Project - Low Level Design

On

IaC Provisioning for Agriculture

Support Triage Agent System

DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	DEEPAK PATRO	EN22CS301316
2.	BHOOMIKA PATIDAR	EN22CS301270
3.	CHARU NARIYA	EN22CS301290
4.	DEV KUMAR DAS	EN22CS301321
5.	AVICHAL MISHRA	EN22CS301243

Group Name:07D3

Project Number:AAI-31

Industry Mentor Name: Mr. Vaibhav

University Mentor Name:Prof. Shyam Patel

Academic Year:2025-2026

Table of Contents

1. Introduction

The *IaC Provisioning for Agriculture Support System* focuses on automating the deployment of the Agriculture AI project using DevOps practices.

Instead of manually setting up the environment, this project uses tools like **Terraform, Docker, Jenkins/GitHub Actions, and Kubernetes** to automatically create and manage infrastructure.

The system ensures that the Agriculture Support System (multi-agent AI project) can be deployed quickly, reliably, and without configuration errors.

2. Scope of the Document

This document provides the **Low Level Design (LLD)** of the DevOps-based deployment system.

It includes:

- Infrastructure setup using IaC
- CI/CD pipeline design
- Docker containerization
- Kubernetes deployment
- AWS/cloud provisioning
- Automation workflow

3. Intended Audience

This document is intended for:

- Developers working on the project
- DevOps engineers

- Students and project reviewers
- Mentors and evaluators

4. System Overview

The system automates deployment of the **Agriculture Support Triage Agent (AI project)**.

Instead of running locally, the system is:

- Containerized using Docker
- Automatically built and deployed using CI/CD
- Hosted on cloud (AWS)
- Managed using Kubernetes

The goal is to make the system:

- Easy to deploy
- Scalable
- Reliable
- Error-free

5. System Design

The system is divided into multiple layers:

1. Source Code Layer

- GitHub repository (your project)
- Contains Python code (Streamlit + CrewAI + RAG)

2. CI/CD Layer

- GitHub Actions / Jenkins
- Automates:
    - Code build

- Docker image creation
- Deployment process

3. Containerization Layer

- Docker
- Converts the Python project into a container

4. Orchestration Layer

- Kubernetes (K8s)
- Manages containers:
  - Scaling
  - Load balancing
  - Deployment

5. Infrastructure Layer (IaC)

- Terraform / Ansible
- Automatically creates:
  - EC2 instances
  - Networking
  - Kubernetes cluster

6. Cloud Layer

- AWS
- Hosts the complete system

6. Application Design

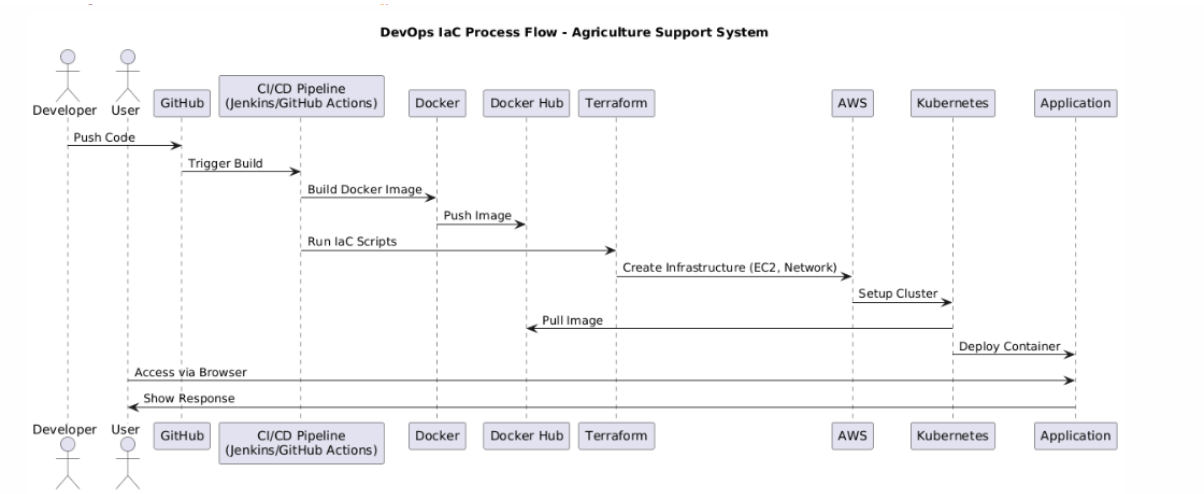
The application is implemented using *Python-based modular architecture*.

Main modules:

Component	Description
GitHub Repo	Stores project code
Dockerfile	Defines container setup
CI/CD Pipeline	Automates build & deployment
Terraform Scripts	Creates infrastructure
Kubernetes YAML	Deploys application
AWS	Cloud hosting

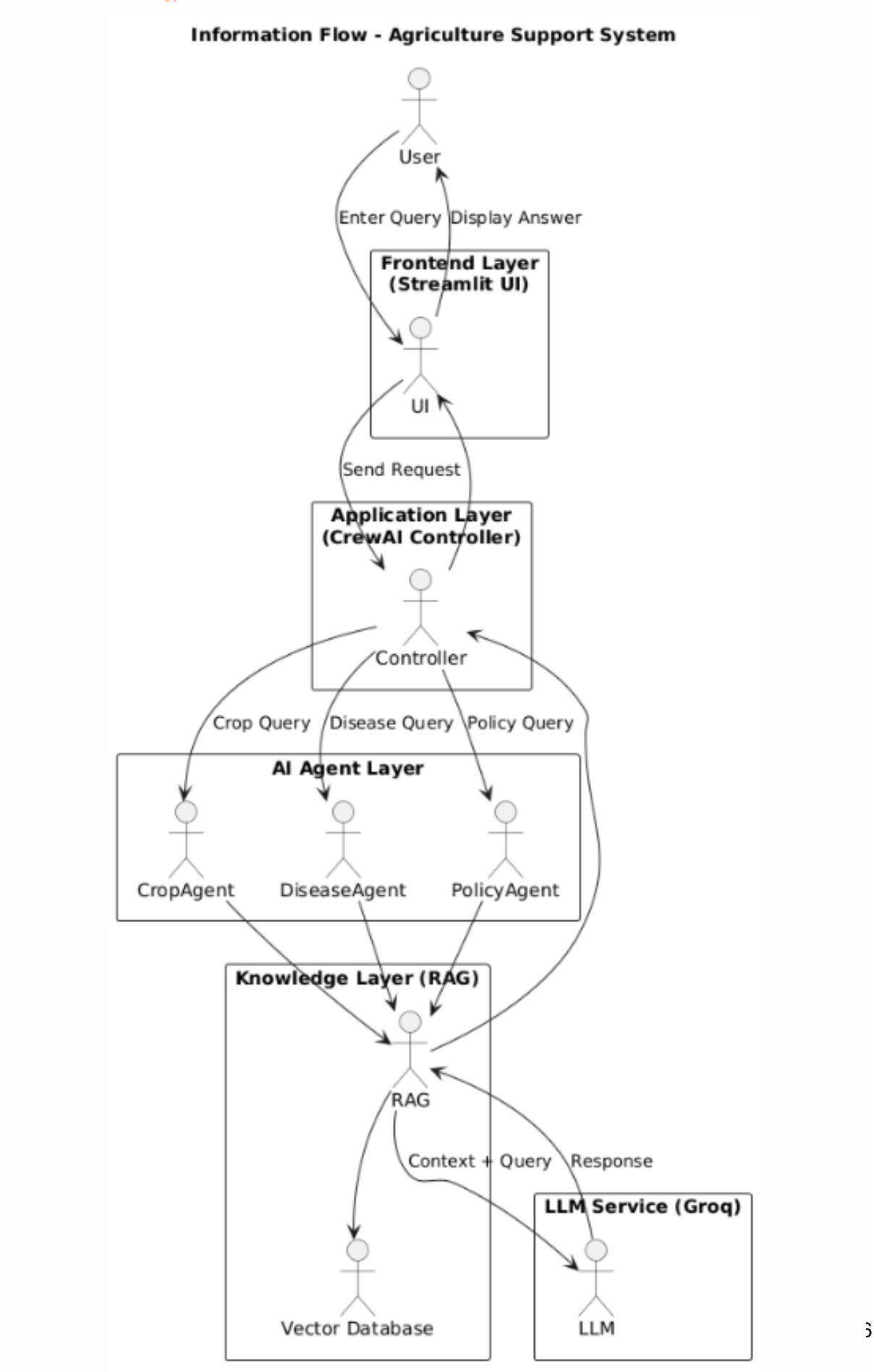
7. Process Flow

The system follows the following process:



8. Information Flow

Information flows through the system in the following sequence:


Information Flow - Agriculture Support System
User
Enter Query
Display Answer
Frontend Layer (Streamlit UI)
UI
Send Request
Application Layer (CrewAI Controller)
Controller
Crop Query
Disease Query
Policy Query
AI Agent Layer
CropAgent
DiseaseAgent
PolicyAgent
Knowledge Layer (RAG)
RAG
Context + Query
Response
LLM Service (Groq)
Vector Database
LLM

9. Components Design

1. GitHub Repository

- Stores complete project code
- Version control

2. Docker

- Packages Python application
- Ensures same environment everywhere

3. CI/CD Pipeline

- Automates:
  - Testing
  - Build
  - Deployment

4. Terraform (IaC Tool)

- Creates AWS infrastructure
- Removes manual setup

5. Kubernetes

- Deploys containers
- Handles scaling and availability

6. AWS Services

- EC2 → compute
- ECR/Docker Hub → image storage
- VPC → networking

10. Key Design Considerations

 Automation
Entire setup is automated using IaC and CI/CD

 Scalability
Kubernetes allows scaling of application

 Reliability
Automated deployment reduces human errors

 Reusability
Same scripts can be reused anytime

 Maintainability
Modular structure makes updates easy.

11. Tools Catalogue

Tools	**Purpose**
GitHub	Code repository
Docker	Containerization
Terraform	Infrastructure provisioning
Kubernetes	Container orchestration
AWS	Cloud hosting
Jenkins / GitHub Actions	CI/CD automation

12. References

 Terraform Documentation

 Docker Documentation

 Kubernetes Documentation

 AWS Documentation

 GitHub Actions Documentation